

Rising Waters: A Machine Learning Approach to Flood Prediction

Project Description

Rising Waters is a Machine Learning-based flood prediction system designed to support early flood risk assessment and disaster preparedness. The system analyzes historical weather and rainfall data to predict the likelihood of flood occurrence in a given region. Users can input environmental parameters such as rainfall intensity, cloud cover, humidity, and seasonal weather conditions through a simple and interactive web interface.

The application leverages the **XGBoost machine learning algorithm** to achieve high accuracy in flood prediction based on trained historical datasets. The backend is developed using **Python and Flask**, while data preprocessing and analysis are performed using **Pandas, NumPy, and Scikit-learn**. The trained model along with the feature scaler is saved using **Joblib** for efficient and fast predictions during runtime.

By providing real-time flood risk assessment, the system aims to enhance disaster preparedness, support decision-making for authorities, and help individuals take timely preventive actions to minimize damage and risk.

Scenarios

Scenario 1: Disaster Management Response

A disaster management officer enters the latest weather parameters, including rainfall, humidity, cloud cover, and river discharge, into the application. The system predicts a **High** or **Severe** flood risk with the corresponding probability, enabling authorities to issue early warnings, coordinate evacuation plans, and allocate emergency resources before flooding occurs.

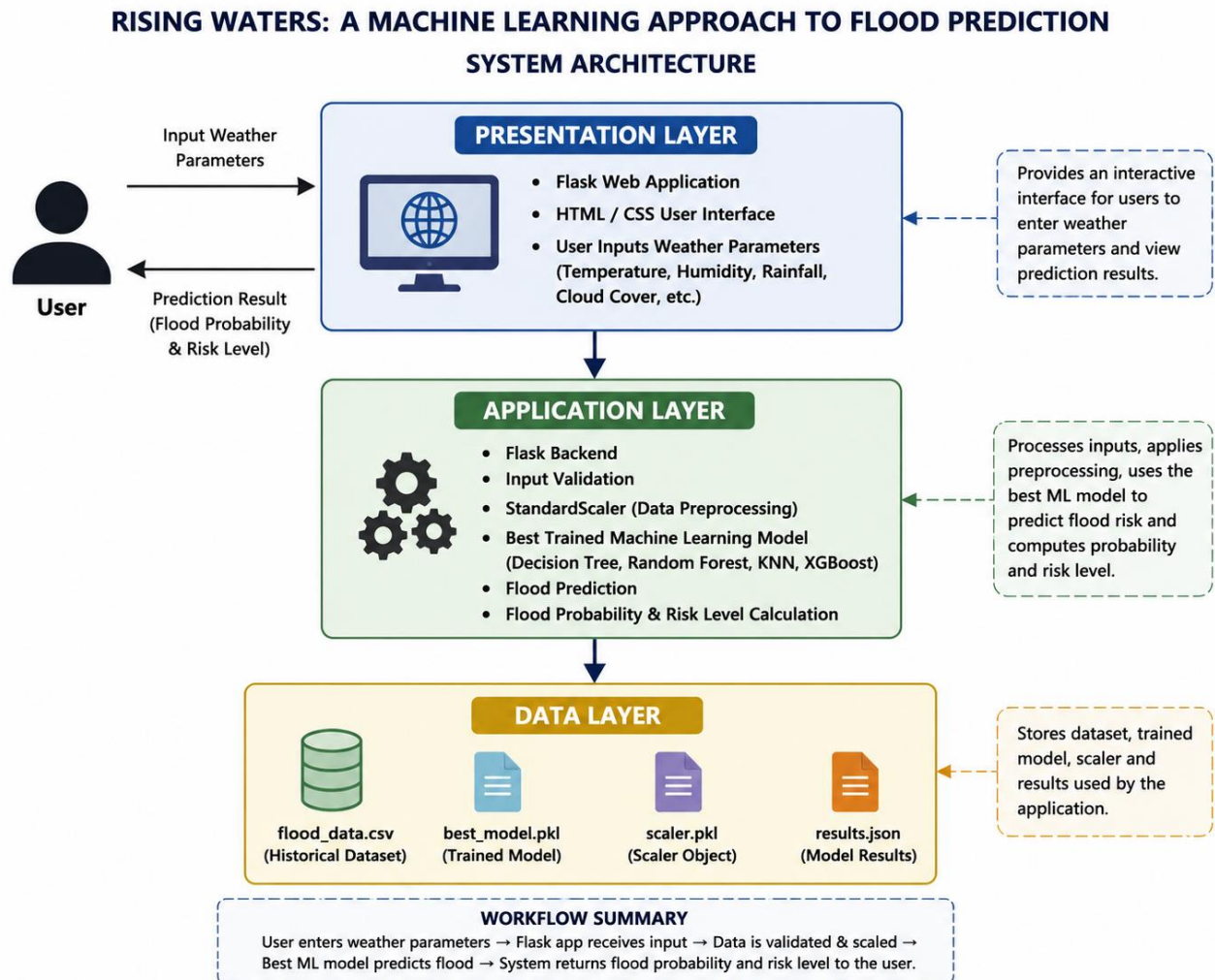
Scenario 2: Weather Monitoring and Planning

A local government official uses the application to monitor weather conditions during the monsoon season. After entering the required environmental parameters, the system predicts a **Low** flood risk, allowing authorities to continue regular monitoring while avoiding unnecessary emergency measures and optimizing resource utilization.

Scenario 3: Community Flood Awareness

A resident living in a flood-prone area accesses the application to check the possibility of flooding based on current weather conditions. The application displays the predicted flood probability and risk level, helping the user make informed decisions such as preparing emergency supplies, protecting valuable belongings, or relocating temporarily if necessary.

Technical Architecture



Description:

Rising Waters follows a **three-layer architecture** consisting of the **Presentation Layer**, **Application Layer**, and **Data Layer**. The **Presentation Layer** is a Flask-based web application developed using **HTML** and **CSS**, providing an interactive interface for users to enter weather parameters and view flood prediction results. The **Application Layer** validates the user input, preprocesses the data using **StandardScaler**, and utilizes the **best-performing trained machine learning model** (selected after comparing Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost) to predict flood probability and determine the corresponding risk level. The **Data Layer** stores the historical weather dataset (**flood_data.csv**), the trained machine learning model (**best_model.pkl**), the preprocessing scaler (**scaler.pkl**), and the model metadata (**results.json**) required for accurate and efficient prediction.

Pre-requisites

- Python Programming
- Flask Framework
- Machine Learning using **Scikit-learn**, **XGBoost**, **Decision Tree**, **Random Forest**, and **K-Nearest Neighbors (KNN)**
- Pandas
- NumPy

- Joblib
- HTML and CSS
- Git and GitHub
- Visual Studio Code (VS Code)

Project Workflow

The development of the **Rising Waters** flood prediction system is organized into five major milestones. Each milestone focuses on a specific phase of the project, starting from dataset preparation and ending with application testing and deployment readiness.

Milestone 1: Dataset Preparation and Environment Setup

Activity 1.1: Prepare the Historical Flood Dataset

The first step involved preparing the historical weather dataset required for machine learning model development. The dataset contains weather and environmental parameters such as annual rainfall, seasonal rainfall, cloud visibility, humidity, river discharge, drainage index, and temperature. The dataset was generated and stored as **flood_data.csv**, which serves as the primary data source for model training and evaluation.

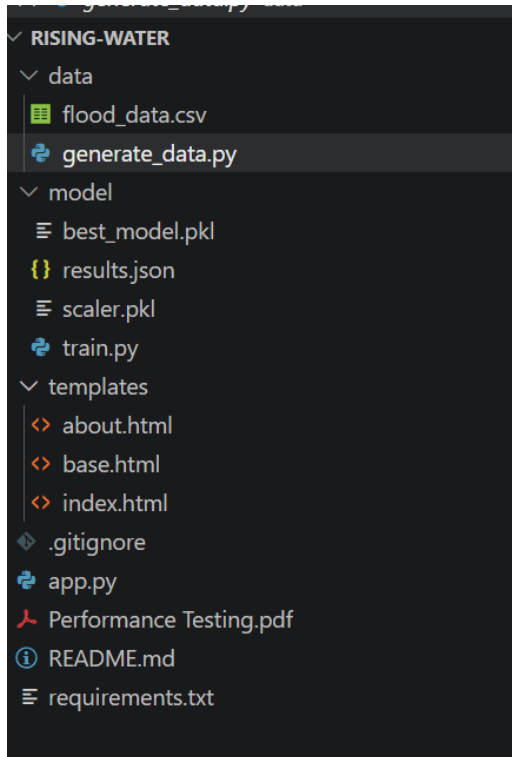
A	B	C	D	E	F	G	H	I	J	K	L
Temp	Humidity	Cloud Cov	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood	
29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.8667	649.9	0	
28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.3	256.4	1	
28	75	42	3271.2	21.7	336.3	2343	570.1	186.2	308.9	0	
29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.0667	862.5	0	
31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.4	586.9	0	
30	70	38	2708	34.1	230	1943.1	500.8	138.3	254.1	0	
29	74	40	3671.1	23.7	328	2737.8	581.7	256.9667	669.5	1	
30	78	36	2648.3	28.8	283.7	2023.6	312.2	197.5333	450	0	
30	71	40	3050.2	65.9	628.3	1940.4	415.5	234.9	231.5	0	
30	70	34	2848.6	28.4	296.7	1886.5	637	226.6667	531.2	0	
28	73	30	2726.7	7.3	249.7	1934	535.7	330	809.4	0	
28	77	40	3451.3	16.9	351.1	2453.1	630.2	316.0667	730.9	1	
28	79	32	2610.8	8.3	295.2	1729	578.3	180.5667	342.9	0	
31	72	35	2899.1	7.6	215	2066.1	610.5	188.4333	401.1	0	
30	73	33	3024.5	40.4	303.1	2167	514	232.0333	541.6	0	
28	76	31	2945.3	7.8	303.4	2170.2	463.9	306.7333	721.2	0	
28	75	31	2704.8	50.5	240.4	1851.7	562.1	234.5667	580.8	0	
31	77	33	2501.9	47.9	767	1104.3	582.6	154.7667	218.7	0	
28	71	43	3003.3	49.2	346.8	2025	582.3	212.2667	389.8	0	
28	77	31	3303.1	40.6	283.7	2318.2	660.6	321.4333	876.6	0	
31	79	43	2719.9	47.8	290.3	1927.5	454.3	163.0333	385	0	
28	74	36	3267.6	51.9	399.4	2231.2	585.1	221.0333	369.5	0	
28	70	30	3484.7	25.3	202.3	2928.4	328.6	240.8333	642.5	1	
30	71	41	4226.4	22.2	363	3451.3	389.9	337.2333	826.3	1	
31	72	36	3062.1	20.5	428.5	1995.2	617.9	229.6	430.6	0	

flood_data



Activity 1.2: Configure the Development Environment

The project development environment was configured using **Python** and **Visual Studio Code (VS Code)**. All required libraries, including **Flask**, **Pandas**, **NumPy**, **Scikit-learn**, **XGBoost**, and **Joblib**, were installed using the **requirements.txt** file to ensure a consistent development environment.



Activity 1.3: Inspect and Understand the Dataset

The dataset was loaded using the Pandas library to verify its structure, feature names, and data quality. Descriptive statistics and basic data inspection were performed to understand the distribution of weather parameters before preprocessing.

Activity 1.4: Design the System Architecture

A three-layer architecture was designed consisting of the **Presentation Layer**, **Application Layer**, and **Data Layer**. This architecture separates the user interface, business logic, and data storage, making the application modular and maintainable.

Milestone 2: Data Preprocessing

Activity 2.1: Data Preparation

The dataset was loaded into the training pipeline, and the required input features and target variable were selected. Data consistency was verified before model training.

```
Dataset shape: (4000, 8)
FLOOD
0    0.62
1    0.38
Name: proportion, dtype: float64
```

	ANNUAL_RAINFALL	SEASONAL_RAINFALL	CLOUD_VISIBILITY	...	DRAINAGE_INDEX	TEMPERATURE	FLOOD
0	2522.9	1502.4	7.77	...	9.73	31.7	0
1	2110.1	1003.6	0.20	...	6.29	30.1	0
2	2621.0	1944.1	10.10	...	8.74	23.4	0
3	3190.0	1931.2	3.79	...	0.01	25.1	1
4	2047.8	1298.8	8.06	...	5.30	26.5	1

```
[5 rows x 8 columns]
```

Activity 2.2: Train-Test Split

The dataset was divided into training and testing datasets using **train_test_split()** with stratified sampling to preserve the class distribution of flood and non-flood instances.

Activity 2.3: Feature Scaling

Feature scaling was performed using **StandardScaler** to normalize the input values. The scaler was fitted using the training dataset and later reused during prediction to maintain consistency between training and inference.

```
df = pd.read_csv("../data/flood_data.csv")
X = df[FEATURES]
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Milestone 3: Machine Learning Model Development

Activity 3.1: Train Multiple Machine Learning Models

Multiple machine learning algorithms were trained and evaluated, including:

- Decision Tree
- Random Forest
- K-Nearest Neighbors (KNN)
- XGBoost

Each model was trained using the preprocessed training dataset to identify the most suitable algorithm for flood prediction.

```
# class imbalance ratio for XGBoost
pos = (y_train == 1).sum()
neg = (y_train == 0).sum()
scale_pos_weight = neg / pos if pos > 0 else 1

models = {
    "Decision Tree": DecisionTreeClassifier(max_depth=5, class_weight="balanced", random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=200, max_depth=6, class_weight="balanced", random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "XGBoost": XGBClassifier(
        n_estimators=150, max_depth=4, learning_rate=0.08,
        subsample=0.9, colsample_bytree=0.9,
        scale_pos_weight=scale_pos_weight,
        eval_metric="logloss", random_state=42
    ),
}
```

Activity 3.2: Model Evaluation

The trained models were evaluated using several performance metrics, including:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

The evaluation results were compared to determine the most effective model.

```
results = {}
trained = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, preds)
    prec = precision_score(y_test, preds, zero_division=0)
    rec = recall_score(y_test, preds, zero_division=0)
    f1 = f1_score(y_test, preds, zero_division=0)
    cm = confusion_matrix(y_test, preds).tolist()
    results[name] = {
        "accuracy": round(acc * 100, 2),
        "precision": round(prec * 100, 2),
        "recall": round(rec * 100, 2),
        "f1_score": round(f1 * 100, 2),
        "confusion_matrix": cm,
    }
    trained[name] = model
    print(f"{name:15s} | Accuracy: {acc*100:.2f}% Precision: {prec*100:.2f}% "
          f"Recall: {rec*100:.2f}% F1: {f1*100:.2f}%")
```

Activity 3.3: Save the Best Model

The model with the highest **F1-score** was selected as the final prediction model. The trained model was saved as **best_model.pkl**, while the preprocessing scaler was saved as **scaler.pkl** using **Joblib**. The evaluation results and feature information were stored in **results.json**.

Model	Accuracy	Precision	Recall	F1 Score
Decision Tree ★	95.65%	100.0%	66.67%	80.0%
Random Forest	95.65%	100.0%	66.67%	80.0%
KNN	86.96%	50.0%	33.33%	40.0%
XGBoost	86.96%	0.0%	0.0%	0.0%

Milestone 4: Flask Web Application Development

Activity 4.1: Develop the Flask Backend

The backend application was developed using **Flask**. The application loads the trained model and scaler, receives user inputs, preprocesses the data, performs prediction, and returns the prediction result.

```
import json
import os
import joblib
import numpy as np
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)

MODEL = joblib.load("model/best_model.pkl")
SCALER = joblib.load("model/scaler.pkl")
with open("model/results.json") as f:
    META = json.load(f)

FEATURES = META["features"]

def build_feature_vector(form):
    """Order form inputs to match training feature order."""
    return [float(form[f]) for f in FEATURES]

@app.route("/")
def home():
    return render_template("index.html", features=FEATURES, meta=META)
```

```
def predict(request):
    """Predict the result based on user input"""
    # Get user input
    temp = request.get_json()
    # Build feature vector
    feature_vector = build_feature_vector(temp)
    # Predict the result
    prediction = MODEL.predict(feature_vector)
    # Return the prediction
    return jsonify({"prediction": prediction})
```

Activity 4.2: Develop the User Interface

A responsive user interface was created using **HTML**, **CSS**, and **Flask Templates**. The application includes the following pages:

- Home Page
- Prediction Page
- About Page

The interface allows users to enter weather parameters and view prediction results in an easy-to-understand format.


Rising Waters | ML Flood Early-Warning System

Predict Model Performance

Flood Risk Predictor

Enter current meteorological readings to estimate flood probability. Powered by the **Decision Tree** model (95.65% test accuracy, 80.0% F1 score).

Temperature (°C)	Humidity (%)
30	75
Cloud Cover (%)	Annual Rainfall (mm)
38	3000
Jan-Feb Rainfall (mm)	Mar-May Rainfall (mm)
20	150
Jun-Sep Rainfall (mm)	Oct-Dec Rainfall (mm)
2200	500

Activity 4.3: Integrate the Machine Learning Model

The Flask application integrates the saved machine learning model with the user interface. Once users submit the required weather parameters, the application preprocesses the inputs, predicts the flood probability, determines the risk level, and displays the final prediction.


Low Flood Risk

Predicted flood probability: **0.0%** | Risk level: **Low**

Model used: Decision Tree

Milestone 5: Testing and Deployment Preparation

Activity 5.1: Local Application Testing

The application was executed locally using Flask. Various input combinations were tested to verify that predictions were generated successfully and displayed correctly.

Activity 5.2: Prediction Verification

Multiple test cases were executed to validate prediction consistency. The application correctly displayed flood probability, risk level, and prediction results for different weather conditions.

Activity 5.3: Deployment Preparation

The project files were organized for deployment. Essential files such as **requirements.txt**, **README.md**, **best_model.pkl**, **scaler.pkl**, and **results.json** were included. The application is currently executed on **localhost** and deployed on **Render**.

 data	Initial commit: flood prediction ML system	18 hours ago
 model	Initial commit: flood prediction ML system	18 hours ago
 templates	Initial commit: flood prediction ML system	18 hours ago
 .gitignore	Initial commit: flood prediction ML system	18 hours ago
 Performance Testing.pdf	Completed Performance Testing and added as a pdf	12 hours ago
 README.md	Revise README for improved project documentation	18 hours ago
 app.py	Prepare for Render deployment	13 hours ago
 requirements.txt	Initial commit: flood prediction ML system	18 hours ago

README



Rising Waters — ML-Powered Flood Prediction System

Rising Waters is a machine learning flood early-warning system trained on historical weather data. It compares four classification algorithms (Decision Tree, Random Forest, K-Nearest Neighbours, and XGBoost), saves the best-performing model, and serves predictions through a Flask web app so disaster management teams and meteorologists can assess flood risk in real time.

